

My Learning Journey — Git, GitHub & Python OOP

by Pascal Musabyimana

When I started these assignments I already had some experience with Git — pushing code, pulling updates, the usual routine. But I quickly realised there was a gap between knowing the commands and actually understanding what Git is doing underneath. These four assignments closed that gap in a way that stuck.

Assignment 1 — The Basics of GitHub

Repo: [git-fundamentals](#)

What I learned

The GitHub Basics course was not difficult, but it was grounding. It forced me to slow down and think about things I had been taking for granted. Git is a *distributed* version control system — meaning every developer has a full copy of the entire history, not just the latest snapshot. That sounds obvious until you really sit with it and realise why it matters: you can work completely offline, roll back to any point in history, and collaborate without a central bottleneck.

What stood out most was the concept of the **GitHub flow** — the rhythm of branching, committing, opening a pull request, getting feedback, and merging. I had done most of these steps before, but always reactively, without thinking of them as a deliberate workflow. After this course I started thinking of a branch not just as a copy of the code, but as a *conversation* with the team about a specific piece of work.

What I delivered

I wrote `my-learning.md` on a dedicated feature branch (`feature/github-learning-notes`), opened a pull request, and merged it into `main` . Even that small exercise made the theory feel real.

Git workflow used

```
git checkout -b feature/github-learning-notes
git add my-learning.md
git commit -m "docs: add GitHub learning notes and course reflection"
git push -u origin feature/github-learning-notes
# → opened PR → merged into main
```

Key concepts I can now explain

- **Repository** — the project folder that tracks all history
- **Clone** — create a local copy of a remote repo, including full history
- **Branch** — an isolated line of work that doesn't affect `main` until merged
- **Commit** — a named checkpoint with a message explaining what changed and why
- **Pull Request** — a proposal to merge a branch, with space for review and discussion
- **Fork** — a personal copy of someone else's repo, used to contribute to open source
- **Issue** — a way to track bugs, tasks, and feature requests

Assignment 2 — Merge Conflict Demo (Basic)

Repo: [merge-demo](#)

What is a merge conflict?

A merge conflict happens when two branches modify the same line of a file differently. Git cannot decide which version is correct, so it stops and asks the developer to resolve it manually. This is not an error — it is Git protecting both sets of changes from being silently overwritten.

The scenario

- `main` changed `h1` to **green**
- `feature-branch` changed `h1` to **red**
- Both branched from the same initial commit where the file was empty

The conflict markers

When `git merge feature-branch` was run on `main`, this appeared in `style.css`:

```
h1 {  
<<<<<<< HEAD  
    color: green;  
    font-size: 24px;  
}  
=====  
    color: red;  
    font-size: 20px;  
}  
>>>>>> feature-branch
```

- `<<<<<<< HEAD` — start of what `main` has
- `=====` — divider between the two versions
- `>>>>>>> feature-branch` — end of what the feature branch has

Resolution

I kept the `main` version (green) and removed all conflict markers:

```
/* style.css */  
h1 {  
    color: green;  
    font-size: 24px;  
}
```

Then staged and committed:

```
git add style.css
git commit -m "Resolve merge conflict in h1 color"
```

Git history after resolution

```
*    fb95ed3 (HEAD -> main) Resolve merge conflict in h1 color
| \
| * bf3432c (feature-branch) Change h1 color to red
* | 4cc5377 Change h1 color to green
* | 9af7485 Add initial website content with blue h1
| /
* f4b931e Initial commit
```

The branching and merging is clearly visible. Both lines of work are preserved in history even though only one version ended up in the final file.

Assignment 3 — Merge Conflict Demo (Advanced E-commerce)

Repo: [merge-demo-project](#)

The scenario

Two teams worked in parallel, both modifying `Product.js` and `pricing.js`:

feature-discount (Team A)

```
// Product.js
class Product {
  calculatePrice() {
    return this.basePrice - this.getDiscount();
  }
}
```

```

    getDiscount() {
      return this.basePrice * this.discountRate;
    }
  }

// pricing.js
export const calculateTotal = (items) => {
  return items.reduce((sum, item) => {
    return sum + item.calculatePrice();
  }, 0);
};

```

feature-tax (Team B)

```

// Product.js
class Product {
  calculatePrice() {
    return this.basePrice + this.getTax();
  }
  getTax() {
    return this.basePrice * this.taxRate;
  }
}

// pricing.js
export const calculateTotal = (items) => {
  return items.reduce((sum, item) => {
    return sum + item.calculatePrice() * (1 + getTax());
  }, 0);
};

```

Why this is harder than the basic example

Both implementations are valid on their own. The conflict is not just technical — it is *logical*. Simply picking one side would silently discard the other team's work.

The correct resolution requires combining both, and in the right order: **discount first, then tax on the discounted price** (standard business logic).

Resolution

```
// Product.js – combined both features
class Product {
  constructor(name, basePrice, discountRate = 0, taxRate = 0)
  {
    this.name = name;
    this.basePrice = basePrice;
    this.discountRate = discountRate;
    this.taxRate = taxRate;
  }

  calculatePrice() {
    const priceWithDiscount = this.basePrice - this.getDiscount();
    return priceWithDiscount + this.getTax(priceWithDiscount);
  }

  getDiscount() {
    return this.basePrice * this.discountRate;
  }

  getTax(priceAfterDiscount) {
    return priceAfterDiscount * this.taxRate;
  }
}

// pricing.js – calculatePrice() already handles both, no duplication needed
export const calculateTotal = (items) => {
  return items.reduce((sum, item) => {
```

```
    const itemPrice = item.calculatePrice();  
    return sum + itemPrice;  
  }, 0);  
};
```

Git history

```
* 85e6254 (HEAD -> main) Resolve merge conflict - combine d  
discount and tax systems  
|\  
| * 30dc962 (feature-tax) Implement tax calculation system  
* | 30b9ab1 (feature-discount) Implement discount system  
|/  
* f9a3998 Initial commit: base Product, Cart, and pricing
```

Key lesson

Resolving a merge conflict sometimes requires a **product decision**, not just a technical one. You need to understand the domain — in this case, standard retail pricing rules — to know which order the operations should run in.

Assignment 4 — Chess (OOP, Decorators, Generators & Git)

Repo: [chess](#)

Project structure

```
chess/  
├─ main.py           # Entry point  
├─ pieces.py         # BaseChessPiece + 6 piece types  
├─ board.py          # Board setup, printing, state savin
```

```
g
└─ board_movements.py    # Static movement helpers
```

1. BaseChessPiece — Inheritance & ABC

`BaseChessPiece` inherits from both `ABC` (Abstract Base Class) and Python's built-in `dict`. The `dict` inheritance is what allows `json.dumps` to serialise each piece automatically when saving board states.

```
from abc import ABC
import functools

class BaseChessPiece(ABC, dict):
    def __init__(self, color: str, name: str, symbol: str, identifier: int):
        self.color = color
        self.name = name
        self.symbol = symbol
        self.identifier = identifier
        self.position = 'None'
        self.is_alive = True
        self.board = None
        # Initialise the dict part for JSON serialisation
        dict.__init__(self, color=color, name=name, symbol=symbol,
                      identifier=identifier, position='None',
                      is_alive=True)

    def die(self):
        self.is_alive = False
        self['is_alive'] = False

    def set_initial_position(self, position: str):
        self.position = position
        self['position'] = position
```



```

def define_board(self, board):
    self.board = board

def __str__(self):
    return f"{self.color} {self.name} {self.identifier}"

def __repr__(self):
    return f"{self.color} {self.name} {self.identifier}"

```

Why inherit from `dict`? When `json.dumps` encounters an object, it checks if it is a `dict` subclass. Because `BaseChessPiece` is one, the board state can be saved with a single `json.dumps(self.squares)` call — no custom serialiser needed.

2. Piece classes — Inheritance in action

Each piece inherits from `BaseChessPiece` and overrides `move()` with its own direction logic. The child's `move()` computes the target square and passes it to `super().move(movement)`, which handles the actual board update.

```

class Pawn(BaseChessPiece):
    def __init__(self, color: str, identifier: int):
        super().__init__(color, 'Pawn', '-', identifier)

    def move(self):
        from board_movements import BoardMovements
        movement = BoardMovements.forward(self.position, self.color, 1)
        super().move(movement)

class Rook(BaseChessPiece):
    def __init__(self, color: str, identifier: int):
        super().__init__(color, 'Rook', 'R', identifier)

    def move(self, direction: str = 'Forward', squares: int =

```

```

1):
    from board_movements import BoardMovements
    directions = {
        'Forward': BoardMovements.forward, 'Backward': Bo
ardMovements.backward,
        'Left': BoardMovements.left,      'Right': Board
Movements.right,
    }
    if direction in directions:
        super().move(directions[direction](self.position,
self.color, squares))

class Knight(BaseChessPiece):
    def __init__(self, color: str, identifier: int):
        super().__init__(color, 'Knight', 'N', identifier)

    def move(self, direction: str = 'ForwardLeft'):
        col_idx = ord(self.position[0]) - ord('a')
        row = int(self.position[1])
        knight_moves = {
            'ForwardLeft': (col_idx - 1, row + 2),
            'ForwardRight': (col_idx + 1, row + 2),
            'BackwardLeft': (col_idx - 1, row - 2),
            'BackwardRight': (col_idx + 1, row - 2),
            'LeftForward': (col_idx - 2, row + 1),
            'LeftBackward': (col_idx - 2, row - 1),
            'RightForward': (col_idx + 2, row + 1),
            'RightBackward': (col_idx + 2, row - 1),
        }
        if direction in knight_moves:
            new_col, new_row = knight_moves[direction]
            if 0 <= new_col <= 7 and 1 <= new_row <= 8:
                super().move(f"{chr(ord('a') + new_col)}{new_
row}")

```

Why `super().__init__(color, 'Pawn', '-', identifier)` ? By calling the parent constructor through `super()`, the Pawn sets all shared properties (color, name, symbol, is_alive, position) in one place. Without this, every piece would have to repeat the same initialisation code.

3. Decorators — @print_board and @save_board

The `@print_board` and `@save_board` decorators are applied to `BaseChessPiece.move()`. Every time any piece moves, the board is automatically printed and the state saved — without any piece needing to know about it.

```
def print_board(func):
    @functools.wraps(func)
    def wrapper(self, *args, **kwargs):
        result = func(self, *args, **kwargs)
        if self.board is not None:
            self.board.print_board()
        return result
    return wrapper

def save_board(func):
    @functools.wraps(func)
    def wrapper(self, *args, **kwargs):
        result = func(self, *args, **kwargs)
        if self.board is not None:
            self.board.save_board_state()
        return result
    return wrapper

# Applied to the base move() method:
@print_board
@save_board
def move(self, movement: str):
    # 1. Check destination
    # 2. Kill enemy if present
    # 3. Update board squares
```

```
# 4. Update self.position
...
```

Execution order: When `pawn.move()` is called → Pawn computes target → calls `super().move(movement)` → `print_board` wrapper runs → `save_board` wrapper runs → actual move executes → state saved → board printed. The Pawn knows nothing about printing or saving.

4. Board setup — Dict comprehension

The board is a dictionary of 64 squares. Using a dict comprehension keeps the initialisation concise and readable:

```
self.squares = {
    f"{chr(col)}{row}": None
    for col in range(ord('a'), ord('i'))    # 'a' through 'h'
    for row in range(1, 9)                 # 1 through 8
}
# Result: {'a1': None, 'a2': None, ..., 'h8': None}
```

Pawns are also placed using dict comprehensions:

```
black_pawns = {
    f"{chr(col)}2": Pawn('BLACK', col - ord('a') + 1)
    for col in range(ord('a'), ord('i'))
}
self.squares.update(black_pawns)
```

5. Board printing — List comprehension

`print_board()` builds the board row by row using a nested list comprehension:

```
def print_board(self):
    rows = [
        [self.squares[f"{chr(col)}{row}"]] for col in range(or
```

```

d('a'), ord('i'))]
    for row in range(1, 9)
]
for row in rows:
    print(row)

```

6. BoardMovements — Static methods

All movement helpers are `@staticmethod` — they don't need `self` or `cls`, they are pure functions. Movement is colour-aware for `forward` and `backward` since BLACK and WHITE move in opposite directions:

```

class BoardMovements:
    @staticmethod
    def forward(position: str, color: str, squares: int = 1)
-> str:
        column = position[0]
        row = int(position[1])
        # BLACK moves toward higher rows, WHITE toward lower
rows
        new_row = row + squares if color == 'BLACK' else row
- squares
        if new_row < 1 or new_row > 8:
            print("Movement blocked: out of bounds")
            return position
        return f"{column}{new_row}"

    @staticmethod
    def forward_left(position: str, color: str, squares: int
= 1) -> str:
        new_pos = BoardMovements.forward(position, color, squ
ares)
        if new_pos == position:
            return position

```

```
result = BoardMovements.left(new_pos, color, squares)
return position if result == new_pos else result
```

7. Generator — Loading board states

Instead of loading the entire `board.txt` file at once (which could be huge after a long game), a generator yields one state at a time:

```
@staticmethod
def load_board_states():
    """Generator: reads board.txt one line at a time."""
    with open('board.txt', 'r') as file:
        for line in file:
            line = line.strip()
            if line:
                yield json.loads(line)
```

Usage in `main.py`:

```
for i, state in enumerate(Board.load_board_states()):
    pieces_on_board = sum(1 for v in state.values() if v is not None)
    print(f"State {i + 1}: {pieces_on_board} pieces on board")
```

Why a generator? A regular function would load all states into memory at once. A generator only processes one line at a time — memory usage stays constant no matter how many states are saved.

8. Running the program — Output

```
=== Basic piece test ===
Created: BLACK Pawn 1

=== Initial board ===
```

```
[BLACK Rook 1, BLACK Knight 1, BLACK Bishop 1, BLACK Queen 1,
BLACK King 1, BLACK Bishop 2, BLACK Knight 2, BLACK Rook 2]
[BLACK Pawn 1, BLACK Pawn 2, BLACK Pawn 3, BLACK Pawn 4, BLAC
K Pawn 5, BLACK Pawn 6, BLACK Pawn 7, BLACK Pawn 8]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[WHITE Pawn 1, WHITE Pawn 2, WHITE Pawn 3, WHITE Pawn 4, WHIT
E Pawn 5, WHITE Pawn 6, WHITE Pawn 7, WHITE Pawn 8]
[WHITE Rook 1, WHITE Knight 1, WHITE Bishop 1, WHITE Queen 1,
WHITE King 1, WHITE Bishop 2, WHITE Knight 2, WHITE Rook 2]
```

```
=== BLACK Pawn 1 moves forward ===
```

```
BLACK Pawn 1 moved to a3
```

```
[BLACK Rook 1, BLACK Knight 1, BLACK Bishop 1, BLACK Queen 1,
BLACK King 1, BLACK Bishop 2, BLACK Knight 2, BLACK Rook 2]
[None, BLACK Pawn 2, BLACK Pawn 3, BLACK Pawn 4, BLACK Pawn
5, BLACK Pawn 6, BLACK Pawn 7, BLACK Pawn 8]
[BLACK Pawn 1, None, None, None, None, None, None, None]
```

```
...
```

```
=== WHITE Pawn 1 moves forward ===
```

```
WHITE Pawn 1 moved to a6
```

```
...
```

```
=== BLACK Knight 1 moves ForwardLeft ===
```

```
Cannot move to a3: occupied by a friendly piece
```

```
...
```

```
=== Replaying saved board states ===
```

```
State 1: 32 pieces on board
```

```
State 2: 32 pieces on board
```

```
State 3: 32 pieces on board
```

Note on colour-aware movement: BLACK Pawn 1 started at `a2` and moved to `a3` (row increased). WHITE Pawn 1 started at `a7` and moved to `a6` (row decreased). Both called the same `forward()` method — the colour parameter handled the direction automatically.

Note on friendly-piece blocking: The Knight attempted `ForwardLeft` from `b1`, which would land on `a3` — already occupied by BLACK Pawn 1. The board correctly blocked the move and printed the reason.

9. Git branch history

```
* b4f8840 (HEAD -> main) chore: update board state log
* f79b0ba docs: add README and initial board state
* 19fd53f (feature/decorators-and-state) feat: add print_board and save_board decorators, generator-based state loading
* d296e83 (feature/board-setup) feat: add board setup, board printing and BoardMovements with all directions
* a36aa9c (feature/chess-pieces) feat: add pawn, rook, bishop, knight, queen and king with base movement stubs
* 96e162f chore: initialize chess assignment structure
```

Each feature was developed on its own branch and merged into `main` — exactly the workflow described in the assignment's Git checkpoints.

Looking Back

These four assignments tell a connected story. I started by understanding the *philosophy* of Git, moved into the *mechanics* of conflict resolution, discovered that good conflict resolution requires *domain knowledge*, and finally built a full OOP application using Git branching as the backbone of the development process.

The thing that surprised me most is how much of software development is about communication — with your team through branches and pull requests, with future readers through commit messages and READMEs, and even with yourself through clean, well-structured code. Git is not just a backup tool. It is a collaboration protocol.

I am looking forward to going deeper — particularly into `git rebase`, GitHub Actions for automated testing, and protected branch rules for team projects. But the foundation is solid now.

Assignment	Repo	Core lesson
GitHub Basics	<u>git-fundamentals</u>	Git is a collaboration protocol, not just a backup tool
Merge Conflict — Basic	<u>merge-demo</u>	Conflicts are Git protecting you, not punishing you
Merge Conflict — Advanced	<u>merge-demo-project</u>	Resolving conflicts requires understanding the domain
Chess OOP	<u>chess</u>	Clean abstractions make complex systems manageable